

EVALUASI TENGAH SEMESTER
MATA KULIAH PEMROGRAMAN KONTROLLER
IMPLEMENTASI PENGENDALI PID DUAL-MODE DENGAN
LOGIKA HISTERESIS PADA SIMULASI MOTOR DC
MENGGUNAKAN STM32



Disusun oleh:

Muhammad Ma'ash Nursalim 2042241012

Satria Mulyawiradija Ramadhan 2042241084

Kelas 4B

Dosen Pengampu:

Ahmad Radhy, S.Si., M.Si

Program Studi Rekayasa Teknologi Instrumentasi
Departemen Teknik Instrumentasi
Institut Teknologi Sepuluh Nopember (ITS)
Semester Genap 2025/2026

1. Studi Literatur

Studi literatur dilakukan terhadap 15 artikel jurnal ilmiah yang terindeks Scopus dan Web of Science (WoS), diterbitkan pada rentang tahun 2021–2026. Jurnal-jurnal tersebut mencakup enam topik utama, yaitu: (a) teknologi dan perkembangan kontroler, (b) dasar mikrokontroler, (c) arsitektur mikrokontroler, (d) periferal (timer, ADC, komunikasi serial), (e) *embedded programming* (C/C++), dan (f) *safety-critical system* pada sistem tertanam.

Table 1: Ringkasan Komprehensif Jurnal Acuan Literatur (2021–2026)

Judul tikel	Ar- Tahun	Penulis & Metode Penelitian	Hasil Utama	Future Work (Saran)
Physical Devices-Agnostic Hybrid Fuzzing of IoT Firmware	Situ, L., et al. (2023)	Hybrid fuzzing menggunakan sistem <i>Virtual Peripheral</i> (VP), <i>symbolic execution</i> , dan mutasi AFL pada core ARM Cortex-M.	Berhasil mengekskusi 88% firmware tanpa hardware fisik; mendeteksi 89.06% kerentanan dalam 24 jam.	Ekspansi arsitektur ke lebih banyak tipe multi-MCU kustom dan otomatisasi periferal.
A Continuous Data Acquisition System for Three-Component Monitoring	Shen, S., et al. (2022)	<i>Hardware-software codesign</i> menggunakan FPGA + STM32, pre-amplifier diferensial, dan 24-bit SAR ADC skema <i>daisy-chain</i> .	Mencapai tingkat <i>self-noise</i> $< 1,32 \mu\text{V}$ pada gain 20 dB serta sinkronisasi clock jaringan $< 1 \mu\text{s}$.	Pengembangan algoritma deteksi otomatis berbasis <i>edge AI</i> dan transmisi nirkabel.
Input Currents of CMOS SAR ADC in Microcontroller	Svatos, J., et al. (2024)	Karakterisasi eksperimental ADC CMOS SAR melalui metode pengosongan kapasitor eksternal dan <i>charge amplifier</i> .	Karakteristik muatan input terbukti nonlinier; nilai C_{SEQ} menyimpang hingga 80–200% dari data <i>datasheet</i> .	Penyusunan model perilaku ADC nonlinier untuk aplikasi IoT berimpedansi tinggi.
Pulse Width Modulation Methods for Commutation Torque Ripples	Lee, J. & Ha, J.-I. (2023)	Metode NSP dan VSP untuk modifikasi <i>duty ratio</i> dan periode <i>switching</i> secara dinamis via <i>Advanced Control Timer</i> STM32.	Mampu mereduksi <i>torque ripple</i> motor dari 40,5% turun menjadi 11,2%.	Penerapan pada sistem BLDC <i>sensorless</i> dan pengujian skema kontrol <i>multi-motor</i> .

Berlanjut ke halaman berikutnya. . .

(Lanjutan Tabel 1)

Judul tikel	Ar- Penulis & Tahun	Metode Penelitian	Hasil Utama	Future Work (Saran)
Microsecond-Level Low-Step Control Platform in M2Plab	Zhou, X., et al. (2025)	Arsitektur <i>multi-task</i> FreeRTOS pada STM32H7, akuisisi ADC + DMA sekuensial, dan <i>numerical solver</i> Runge-Kutta 4 (RK4).	Eksekusi GPIO toggle presisi setiap 25 μ s; efisiensi pre-kompilasi hemat 86,7%.	Dukungan platform <i>cloud</i> universal dan perluasan pustaka blok kendali.
Support of MISRA C++ Analyzer for Reliability	Lin, C.-C., et al. (2025)	Pengembangan <i>rule checker</i> menggunakan infrastruktur Clang LLVM berbasis traversal AST.	158 Kesesuaian 100% pada pengujian <i>diamond inheritance</i> ; deteksi defect lebih baik dari Coverity.	Analisis pointer menggunakan LLVM Memory SSA dan perbaikan kode otomatis.
Real-Time Operating Systems' Compliance With MISRA-C	Song, J., et al. (2024)	<i>Static Code Analysis</i> via Cppcheck untuk mengukur kepatuhan 16 varian RTOS <i>open-source</i> terhadap 153 aturan.	14 dari 16 RTOS patuh total terhadap aturan <i>mandatory</i> ; pelanggaran kode hanya 6,90%.	Penyusunan <i>automated patch/fix tool</i> dan analisis RTOS berbasis bahasa Rust.
Real-Time Design Patterns for Safety-Critical Systems	Mzid, R. (2024)	Penerapan <i>Model-Driven Engineering</i> (MDE) berbasis UML profile dan verifikasi formal analisis <i>schedulability</i> (CHEDDAR).	Pola katalog P2VRT sukses menstandarisasi batas durasi <i>deadline</i> task kritis secara <i>real-time</i> .	Perluasan pola ke sistem tanam dengan aktivasi <i>aperiodik</i> dan integrasi AI.
Benefits of Using Design Patterns on Microcontrollers	Babiuch, M. & Foltynnek, P. (2024)	<i>Universal modular framework</i> menerapkan prinsip SOLID dan pola arsitektur <i>Sensor-View</i> terinspirasi MVC.	Logika aplikasi terisolasi penuh dari <i>driver</i> fisik; ganti sensor tanpa mengubah kode inti.	Kerangka kerja modular <i>multi-platform</i> untuk integrasi sistem berskala luas.
Global Optimization in Robust Fractional Control	Rhouma, A., et al. (2022)	<i>Robust Fractional Predictive Control</i> (RFPC) berbasis konversi optimasi <i>non-convex</i> ke <i>convex</i> menggunakan GGP.	GGP terbukti menjamin solusi optimasi global pada sistem termal secara <i>real-time</i> .	Pengujian algoritma optimasi pada arsitektur MIMO yang lebih cepat.

Berlanjut ke halaman berikutnya...

(Lanjutan Tabel 1)

Judul tikel	Ar- Penulis & Tahun	Metode Penelitian	Hasil Utama	Future Work (Saran)
FPGA Integrated 802.15.4 ZigBee Wireless Nodes	Inte- IEEE (2022) Ompal, et al.	Sintesis arsitektur <i>hardware</i> simpul sensor nirkabel berbasis ZigBee pada FPGA Xilinx Virtex-5.	Latensi minimum 9,1 ns dan konsumsi daya operasional 7,5 mW.	Integrasi <i>Machine Learning</i> di sisi <i>edge node</i> untuk deteksi anomali transmisi.
Implications of Memory Embedding and Hierarchy	Chowdhury, M. & Rahman, M. (2025)	Evaluasi hierarki memori menggunakan simulator <i>cycle-accurate</i> antara eDRAM vs SRAM <i>on-chip</i> .	eDRAM + HBM2 meningkatkan akselerasi AI sebesar 4,28 kali lipat.	Implementasi <i>compiler data orchestration</i> adaptif untuk reduksi latensi internal.
A Smart Microcontroller Architecture for the IoT	Wu, Z., et al. (2021)	Pemodelan arsitektur cerdas berlayer: sub-sistem <i>sensing, communication, control</i> , dan <i>application</i> .	Implementasi pada perangkat pengunci RFID memotong durasi <i>maintenance</i> hingga 35%.	Sistem <i>edge processing</i> ringan untuk pengambilan keputusan lokal.
Frequency and Real-Time Properties of FOPID Controller	Oprzedkiewicz, K., et al. (2021)	Implementasi kontroler fraksional FOPID pada STM32 menggunakan metode komparatif CFE dan FOBD.	Aproksimasi CFE memenuhi target <i>soft real-time</i> pada mikro berkapasitas memori terbatas.	Pengujian <i>hard real-time</i> menggunakan <i>timer interrupt</i> atau RTOS rigid.
Sensing Power Consumption on Motion Profiles	Olmedo-Garcia, L., et al. (2023)	Analisis profil gerakan mekanis (<i>Parabolic, Trapezoidal, S-Curve</i>) berbasis sensor arus dan <i>trajectory generator</i> .	Profil <i>parabolic</i> menghasilkan efisiensi energi 22,19% lebih hemat.	Sistem kendali adaptif pemilih profil berdasarkan dinamika perubahan besar beban.

1.1. Daftar *Future Work* yang Diintegrasikan

Berikut adalah ringkasan *future work* (FW) spesifik yang disintesis menjadi dasar pengembangan platform **AIFRIS**:

1. **FW1 (Jurnal 1):** Otomasi analisis dan konfigurasi periferal kustom tanpa dependensi hardware fisik.
2. **FW2 (Jurnal 2):** Algoritma deteksi *event* otomatis berbasis *edge computing* langsung di *embedded device*.

3. **FW3 (Jurnal 3):** Model perilaku ADC dengan koreksi C_{SEQ} adaptif untuk kompensasi nonlinearitas pada sumber impedansi tinggi.
4. **FW4 (Jurnal 4):** Sinkronisasi periode PWM dengan *timer interrupt* untuk eliminasi delay komutasi pada kontrol aktuator.
5. **FW5 (Jurnal 5):** Dukungan multi-MCU dengan library blok kontrol beragam dan monitoring *real-time* berbasis jaringan.
6. **FW6 (Jurnal 6):** Integrasi *static code checker* MISRA-C/C++ ke dalam pipeline pengembangan untuk menjamin keandalan kode.
7. **FW7 (Jurnal 7):** *Automated patch/fix tool* MISRA-C dan integrasi CI/CD untuk pengembangan RTOS yang lebih aman.
8. **FW8 (Jurnal 8):** Rekomendasi *design pattern real-time* otomatis berbasis AI untuk sistem *embedded safety-critical*.
9. **FW9 (Jurnal 9):** *Framework* modular multi-platform untuk pemisahan driver sensor dari logika aplikasi pada IoT.
10. **FW10 (Jurnal 10):** Metode aproksimasi numerik ringan untuk implementasi kontroler fraksional pada MCU terbatas.
11. **FW11 (Jurnal 11):** Enkripsi data dan integrasi ML untuk deteksi anomali langsung di *edge node*.
12. **FW12 (Jurnal 12):** *Data orchestration* adaptif untuk meminimalkan latensi perpindahan data antar-memori.
13. **FW13 (Jurnal 13):** *Edge processing* ringan dan monitoring IoT *real-time* langsung tanpa server eksternal.
14. **FW14 (Jurnal 14):** Pengujian *hard real-time* via *timer interrupt* dengan pengukuran waktu eksekusi.
15. **FW15 (Jurnal 15):** Sistem kontrol adaptif pemilih profil kontrol berdasarkan besar *error* dan konsumsi energi aktuator.

2. Metode yang Diusulkan

2.1. Deskripsi Umum Sistem Kontrol

Penelitian ini mengusulkan implementasi sistem kontrol PID dua mode (dual-mode PID) pada motor DC yang dikontrol oleh mikrokontroler STM32F103C8T6. Sistem dirancang menggunakan pendekatan Software-in-the-Loop (SIL) simulation di lingkungan Proteus untuk validasi algoritma sebelum implementasi pada hardware fisik.

Tujuan utama adalah merancang algoritma kontrol yang dapat merespons cepat saat error besar (mode agresif) namun tetap stabil saat mendekati setpoint (mode normal), sehingga mengatasi trade-off fundamental pada PID tunggal.

2.2. Justifikasi Pemilihan Aksi Kontrol: Mengapa PID?

Pemilihan aksi kontrol yang tepat harus didasarkan pada karakteristik dinamika plant (motor DC) yang akan dikendalikan. Motor DC memiliki tiga karakteristik penting yang mempengaruhi desain controller:

1. **Gesekan Statis (Static Friction):** Motor DC tidak akan berputar jika tegangan input lebih kecil dari batas gesekan statis tertentu. Hal ini menyebabkan sistem memiliki dead-zone, di mana error kecil tidak menghasilkan aksi kontrol yang cukup untuk menggerakkan motor.
2. **Inersia Mekanis (Mechanical Inertia):** Motor memiliki kelembaman rotasi yang menyebabkan sistem merespons lambat terhadap perubahan sinyal kontrol. Ini menciptakan dinamika order-1 atau order-2 yang memperlambat settling time.
3. **Steady-State Error pada Kontrol P Murni:** Proportional-only controller tidak mampu menghilangkan steady-state error yang disebabkan oleh gesekan statis. Error yang tersisa akan menghasilkan sinyal kontrol konstan yang tidak cukup untuk menggerakkan motor.

Table 2: Perbandingan Aksi Kontrol P, PI, dan PID untuk Motor DC

Kriteria	P	PI	PID
SSE Permanen	Ada	Dihilangkan	Dihilangkan
Overshoot	Rendah	Sedang-Tinggi	Rendah
Rise Time	Cepat	Lambat	Cepat
Stabilitas	Kurang	Baik	Baik
Cocok untuk Motor DC	Tidak	Kurang Ideal	Ya

Alasan Memilih PID Penuh:

- **Aksi I (Integral):** Diperlukan untuk menghilangkan steady-state error yang disebabkan oleh gesekan statis motor. Integral mengakumulasi error kecil hingga menghasilkan sinyal kontrol yang cukup untuk melawan gesekan.
- **Aksi D (Derivative):** Diperlukan sebagai fungsi redaman (damping) untuk meredam overshoot yang mungkin terjadi akibat inersia motor. Derivative term membaca laju

perubahan error dan memberikan peredaman saat error berkurang cepat.

- **Aksi P (Proportional):** Memberikan respons awal (proportional gain) yang cepat terhadap perubahan error.

Dengan kombinasi ketiga aksi ini, PID mampu menghasilkan respons yang cepat, stabil, dan tanpa steady-state error — karakteristik ideal untuk kontrol motor DC.

Referensi: Ogata (2010, *Modern Control Engineering*, Ch. 3) membahas pemilihan aksi kontrol berdasarkan karakteristik plant. Seborg et al. (2017, *Process Dynamics and Control*, Ch. 9) memberikan panduan praktis untuk tuning PID pada sistem dengan dead-zone.

2.3. Model Matematis Plant Motor DC

2.3.1 Persamaan Diferensial Motor DC

Untuk merancang dan mensimulasikan sistem kontrol, diperlukan model matematis yang merepresentasikan dinamika motor DC. Model yang digunakan adalah motor DC orde-1 yang umum digunakan pada aplikasi sederhana dan medium.

Dinamika motor DC dideskripsikan oleh dua persamaan fundamental:

Persamaan Elektrik:

$$V(t) = i(t) \cdot R + L \frac{di(t)}{dt} + K_e \cdot \omega(t) \quad (1)$$

Persamaan Mekanik:

$$J \frac{d\omega(t)}{dt} = K_t \cdot i(t) - b \cdot \omega(t) \quad (2)$$

Untuk aplikasi sederhana seperti pengujian pada simulator Proteus, model di atas dapat disederhanakan menjadi sistem orde-1 dengan mengasumsikan induktansi L diabaikan dan tidak ada torsi beban eksternal.

Model orde-1 yang dihasilkan adalah:

$$\tau \frac{d\omega(t)}{dt} + \omega(t) = K \cdot u(t) \quad (3)$$

Transformasi ke domain s (Laplace):

$$G(s) = \frac{\omega(s)}{U(s)} = \frac{K}{\tau s + 1} \quad (4)$$

Parameter yang digunakan dalam simulasi: $\tau = 0.2$ s (time constant 200 ms), $K = 1.0$ (normalized gain).

Referensi: Franklin et al. (2015, *Feedback Control of Dynamic Systems*, Ch. 2) memberikan derivasi lengkap model motor DC dari prinsip fisika dasar.

2.4. Algoritma PID Dual-Mode dengan Hysteresis

2.4.1 Formulasi Umum PID Diskrit

Persamaan PID dalam domain waktu diskrit (implementasi pada mikrokontroler):

$$u(k) = K_p \cdot e(k) + K_i \sum_{j=0}^k e(j) \cdot T_s + K_d \frac{e(k) - e(k-1)}{T_s} \quad (5)$$

di mana: $e(k) = r(k) - y(k)$ adalah error pada waktu sampling k , $T_s = 100$ ms adalah sampling time, dan $u(k)$ adalah control output yang dibatasi dalam range 0–100% (PWM duty cycle).

Referensi: Franklin et al. (2015, Ch. 3) dan Kuo & Golnaraghi (2017, *Automatic Control Systems*, Ch. 8).

2.4.2 Mekanisme Dual-Mode dan Switching Logic

Sistem menggunakan dua set parameter PID yang dipilih berdasarkan magnitude error:

Mode Normal (Stabil): $K_p = 1.50$, $K_i = 0.70$, $K_d = 0.05$

Mode Agresif (Responsif): $K_p = 2.20$, $K_i = 0.30$, $K_d = 0.08$

Logika Switching dengan Hysteresis:

$$\text{Mode} = \begin{cases} \text{AGRESIF} & \text{jika } |e(k)| > 12\% \\ \text{NORMAL} & \text{jika } |e(k)| < 4.5\% \\ \text{Mode saat ini} & \text{jika } 4.5\% \leq |e(k)| \leq 12\% \end{cases} \quad (6)$$

Mekanisme hysteresis (zona mati antara 4.5% dan 12%) dirancang untuk mencegah chattering, yaitu perpindahan mode yang terlalu cepat yang dapat menyebabkan sistem tidak stabil.

Nilai-nilai parameter PID di atas ditentukan melalui metode trial-and-error sistematis yang dipandu oleh teori tuning PID standar (Ziegler-Nichols) dengan fine-tuning eksperimental pada empat skenario pengujian.

Referensi: Ziegler & Nichols (1942, *Optimum Settings for Automatic Controllers*) dan Åström & Hägglund (2006, *Advanced PID Control*).

2.5. Arsitektur Sistem Kontrol (Diagram Blok)

Arsitektur interkoneksi sistem kendali ditunjukkan pada Gambar 1. Sistem terdiri dari tiga layer utama:

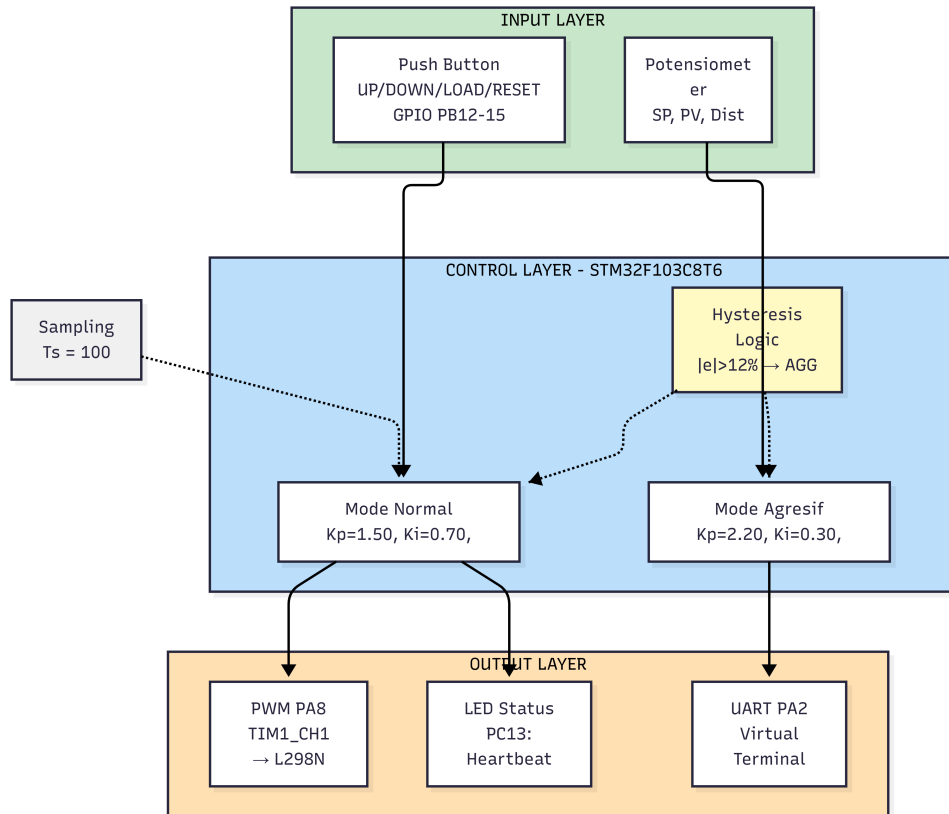


Figure 1: Diagram Blok Sistem Kontrol Motor DC Dual-Mode PID pada STM32F103C8T6.

1. **Input Layer:** Akuisisi sinyal dari tombol (button UP/DOWN/LOAD/RESET) untuk mengatur setpoint
2. **Control Layer:** Algoritma PID dual-mode pada STM32F103C8 dengan mekanisme hysteresis untuk switching mode
3. **Output Layer:** Output sinyal PWM ke motor driver L298N, telemetri via UART ke Virtual Terminal, dan LED status

Sistem beroperasi pada loop kontrol tertutup (closed-loop control) dengan sampling time 100 ms. Setiap 100 ms, sistem membaca error, menentukan mode berdasarkan logika hysteresis, menghitung output PID sesuai parameter mode yang aktif, mengirimkan PWM ke motor driver, dan mengirimkan data telemetri via UART.

2.6. Alur Algoritma Sistem (Flowchart)

Flowchart pada Gambar 2 menunjukkan alur logika algoritma kontrol secara sekuensial. Proses dimulai dengan inisialisasi sistem, dilanjutkan dengan pembacaan input (button dan ADC), perhitungan error, penentuan mode berdasarkan magnitude error, perhitungan output PID, clamping output ke range 0–100%, pengiriman sinyal PWM, transmisi data UART, delay 100 ms, dan loop kembali ke pembacaan input.

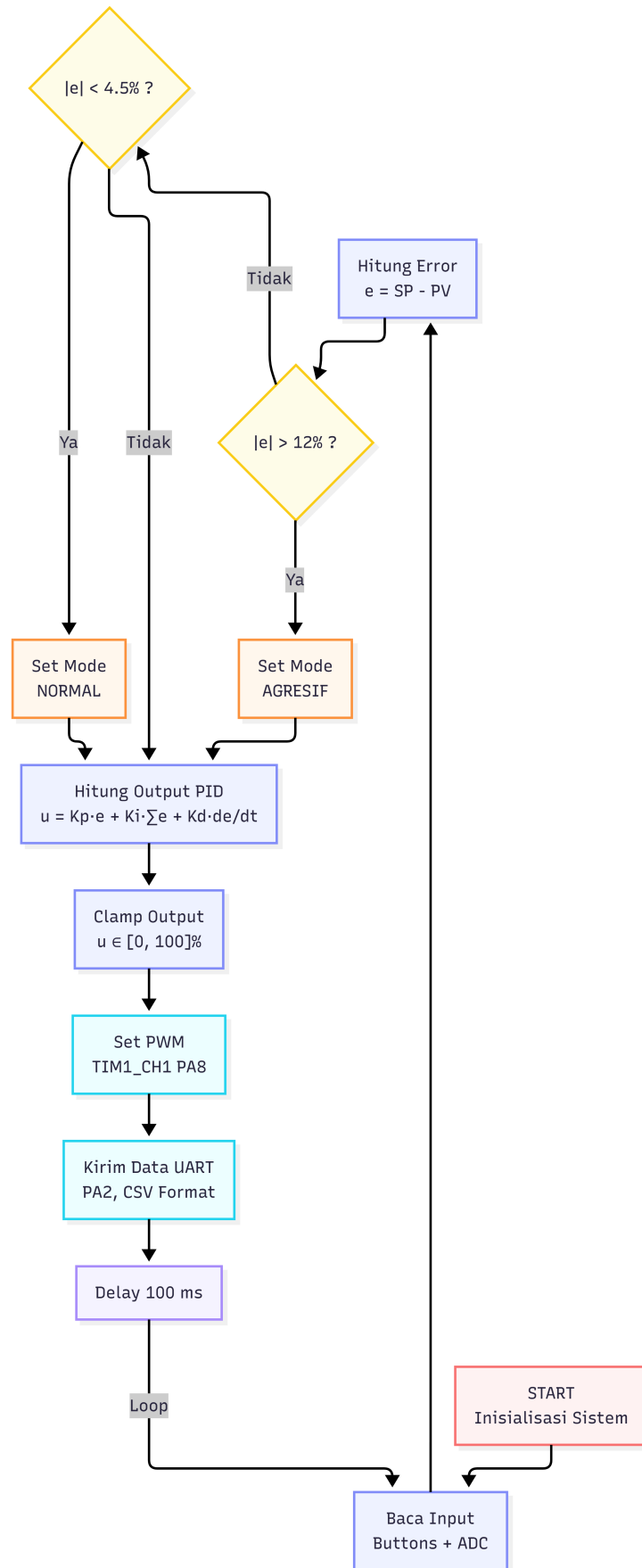


Figure 2: Flowchart Algoritma Kontrol PID Dual-Mode dengan Hysteresis.

2.7. Metode Validasi dan Pengujian

Untuk memvalidasi kebenaran algoritma dan kinerja sistem, dilakukan pengujian pada empat skenario operasional yang mewakili kondisi nyata penggunaan motor DC:

1. **Skenario A — Step Response:** Setpoint dinaikkan secara bertahap ($0\% \rightarrow 10\% \rightarrow 20\% \rightarrow 30\%$) untuk mengamati respons transien sistem, termasuk rise time, overshoot, dan settling time.
2. **Skenario B — Tracking Test:** Setpoint dijaga konstan pada 30% untuk mengamati kemampuan sistem mempertahankan target dalam jangka panjang (steady-state error).
3. **Skenario C — Disturbance Rejection:** Dilakukan injeksi gangguan (disturbance) eksternal untuk menguji ketangguhan sistem dalam mengatasi gangguan.
4. **Skenario D — Hysteresis Validation:** Setpoint dinaikkan drastis ($0\% \rightarrow 50\%$) untuk mengamati mekanisme hysteresis dan mode switching.

Data hasil pengujian dikumpulkan via telemetri UART dalam format CSV dan divisualisasikan menggunakan GNUplot untuk analisis kuantitatif. Hasil pengujian akan dibahas secara detail pada Bab 4.

3. Rangkaian Simulasi dan Metodologi Pengujian

3.1. Pendekatan Simulasi Software-in-the-Loop (SIL)

Untuk memvalidasi algoritma kontrol PID dual-mode dan mekanisme switching hysteresis, dilakukan simulasi menggunakan pendekatan Software-in-the-Loop (SIL) di lingkungan Proteus. Metode SIL dipilih karena memungkinkan pengujian algoritma kontrol secara menyeluruh tanpa memerlukan hardware fisik pada tahap awal pengembangan.

Dalam simulasi SIL, model matematis plant motor DC (persamaan transfer function orde-1 yang dijelaskan pada Bab 2) diimplementasikan langsung dalam firmware mikrokontroler. Dengan demikian, meskipun motor fisik di Proteus tidak menampilkan gerakan secara visual, data telemetri yang dihasilkan sistem (error, control output, mode switching) tetap valid dan dapat digunakan untuk validasi algoritma.

Pendekatan ini umum digunakan dalam industri embedded systems untuk mempercepat siklus pengembangan dan mengurangi risiko kerusakan hardware pada tahap prototyping.

3.2. Perancangan Rangkaian Hardware

Rangkaian simulasi dirancang untuk merepresentasikan sistem kontrol motor DC secara lengkap, mencakup mikrokontroler, motor driver, aktuator, interface pengguna, dan monitoring tools. Komponen-komponen yang digunakan dalam simulasi Proteus adalah:

Table 3: Daftar Komponen Rangkaian Simulasi Proteus

Komponen	Jumlah	Fungsi
STM32F103C8T6	1	Mikrokontroler utama untuk menjalankan algoritma PID dual-mode
L298N Motor Driver	1	H-Bridge driver untuk menggerakkan motor DC berdasarkan sinyal PWM
DC Motor	1	Aktuator (plant) yang dikontrol oleh sistem
Push Button	4	Input untuk mengatur setpoint (UP, DOWN, LOAD, RESET)
LED	3	Indikator status: Heartbeat (PC13), Mode Normal (PC14), Mode Agresif (PC15)
Virtual Terminal	1	Monitor data telemetri UART dalam format CSV
Oscilloscope	1	Monitor sinyal PWM pada pin PA8 (TIM1_CH1)

Gambar 3 menunjukkan skematik rangkaian lengkap yang diimplementasikan pada simulator Proteus. Rangkaian ini mencakup seluruh komponen yang diperlukan untuk simulasi closed-loop control system, mulai dari input (push button), processing (STM32), actuator (motor DC via L298N), hingga monitoring tools (Virtual Terminal dan Oscilloscope).

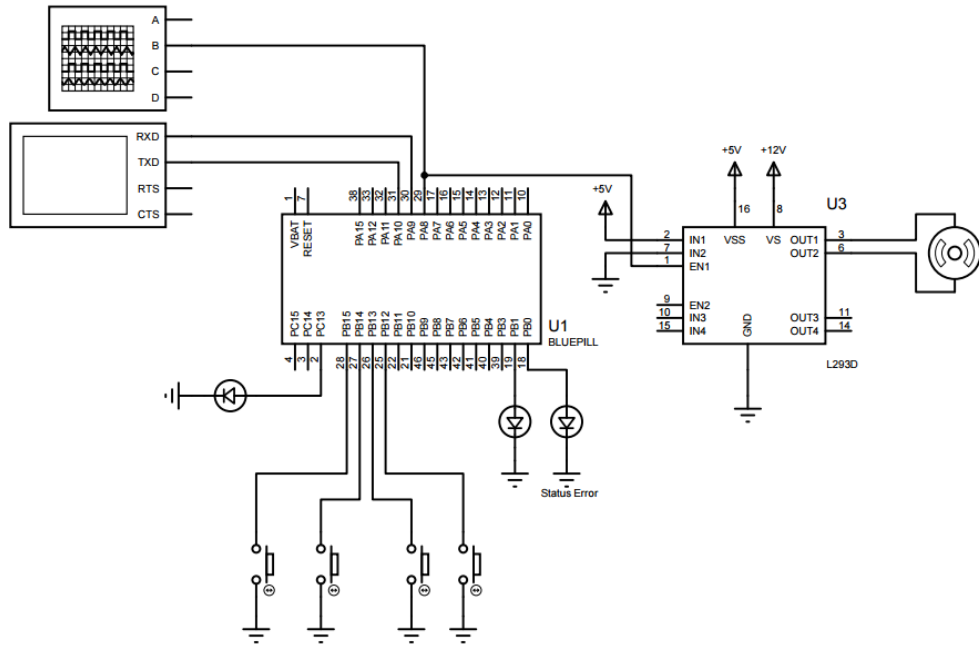


Figure 3: Skematik Rangkaian Lengkap Sistem Kontrol Motor DC di Proteus.

3.3. Pemetaan Pin Mikrokontroler (Pin Mapping)

Konfigurasi pin STM32F103C8T6 dirancang berdasarkan fungsi alternate function dari setiap peripheral yang digunakan. Pemetaan pin yang digunakan dalam sistem adalah sebagai berikut:

Table 4: Pemetaan Pin STM32F103C8T6

Pin	Fungsi	Keterangan
PA8	TIM1_CH1 (PWM)	Output PWM ke pin ENA motor driver L298N
PA9	GPIO Output	Output IN1 motor driver (arah putar 1)
PA10	GPIO Output	Output IN2 motor driver (arah putar 2)
PA2	USART2_TX	Transmit data telemetri CSV ke Virtual Terminal
PB12	GPIO Input	Push button UP (increment setpoint)
PB13	GPIO Input	Push button DOWN (decrement setpoint)
PB14	GPIO Input	Push button LOAD (apply setpoint)
PB15	GPIO Input	Push button RESET (reset sistem)
PC13	GPIO Output	LED Heartbeat (indikator sistem hidup)
PC14	GPIO Output	LED Mode Normal (hijau)
PC15	GPIO Output	LED Mode Agresif (merah)

Pemilihan pin didasarkan pada alternate function yang tersedia pada STM32F103C8T6. Pin PA8 dipilih untuk PWM karena merupakan channel 1 dari Advanced Timer TIM1 yang memiliki resolusi tinggi. Pin PA2 dipilih untuk UART karena merupakan TX dari USART2 yang umum digunakan untuk komunikasi serial.

Referensi: STM32F103C8T6 Datasheet (STMicroelectronics, 2023) dan Reference Man-

ual RM0008 (STMicroelectronics, 2023) memberikan informasi lengkap tentang alternate function dan konfigurasi GPIO.

3.4. Karakteristik Simulasi Software-in-the-Loop

3.4.1 Implementasi Model Plant di Firmware

Dalam simulasi SIL, dinamika plant motor DC tidak disimulasikan oleh komponen motor fisik di Proteus, melainkan diimplementasikan secara matematis dalam firmware mikrokontroler. Model plant yang digunakan adalah transfer function orde-1 yang telah dijelaskan pada Bab 2:

$$G(s) = \frac{K}{\tau s + 1}$$

Model ini di-diskritisasi menggunakan metode Euler forward dengan sampling time $T_s = 100$ ms dan diimplementasikan dalam fungsi C sebagai berikut:

```
float simulatePlant(float pwm_input, float prev_position) {
    float tau = 0.2; // time constant 200 ms
    float K = 1.0;   // static gain
    float Ts = 0.1;  // sampling time 100 ms

    // Euler forward discretization
    float new_position = prev_position + (Ts/tau) *
                                (K * pwm_input - prev_position);
    return new_position;
}
```

Dengan implementasi ini, sistem dapat menghasilkan data telemetri yang valid meskipun motor fisik di Proteus tidak bergerak secara visual. Data yang dihasilkan (position, error, control output) mencerminkan behavior sistem kontrol yang sebenarnya dan dapat digunakan untuk analisis performa.

3.4.2 Validitas Data Telemetri

Meskipun motor fisik di Proteus tidak menampilkan gerakan visual, data telemetri yang dihasilkan sistem tetap valid karena:

1. **Model Plant Matematis:** Plant di-simulasi menggunakan persamaan diferensial yang merepresentasikan dinamika motor DC secara akurat.
2. **Algoritma Kontrol Real:** Algoritma PID dual-mode yang dijalankan di mikrokontroler adalah kode yang sama dengan yang akan digunakan pada hardware fisik.
3. **Timing Real:** Sampling time, delay, dan interrupt timing yang digunakan dalam simulasi sama dengan kondisi real hardware.

Pendekatan SIL ini umum digunakan dalam industri untuk validasi algoritma kontrol sebelum implementasi pada hardware, dan telah terbukti efektif untuk mengurangi waktu dan biaya pengembangan.

Referensi: Hanselmann (1993, *Hardware-in-the-Loop Simulation as a Standard Approach for Development, Customization, and Production Test*) dan Isermann et al. (1999, *Hardware-in-the-loop simulation for the design and testing of engine-control systems*) membahas validitas metode HIL/SIL dalam pengembangan sistem embedded.

3.5. Monitoring dan Instrumentasi

3.5.1 Virtual Terminal untuk Telemetri Data

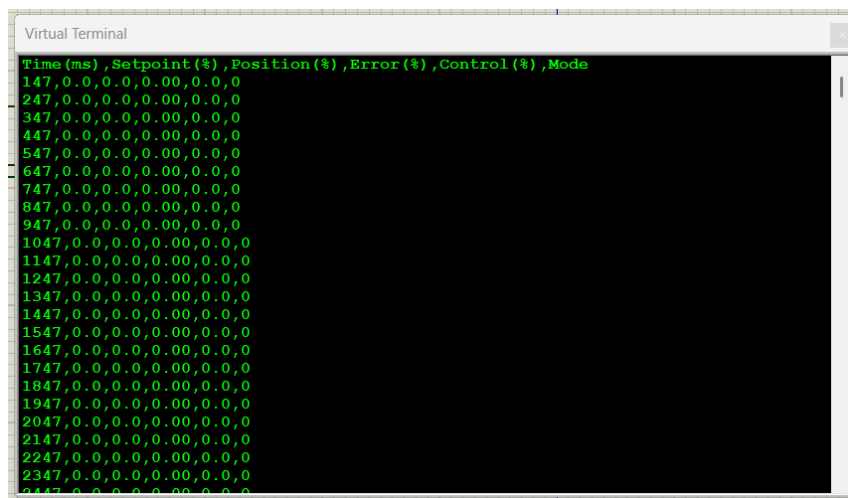


Figure 4: Output Virtual Terminal Menampilkan Data Telemetri dalam Format CSV.

Data telemetri dari mikrokontroler dikirimkan secara periodik (setiap 100 ms) melalui UART dengan baud rate 115200 bps dalam format CSV (Comma-Separated Values). Format data yang dikirimkan adalah:

```

Time(ms),Setpoint(%),Position(%),Error(%),Control(%),Mode
147,0.0,0.0,0.00,0.0,0
247,0.0,0.0,0.00,0.0,0
347,10.0,3.1,6.86,9.7,0
...

```

di mana:

- Time(ms): Waktu sejak sistem start (milliseconds)
- Setpoint(%): Target posisi yang diinginkan (0–100%)
- Position(%): Posisi aktual motor hasil simulasi plant
- Error(%): Selisih antara setpoint dan position
- Control(%): Output kontrol PID (PWM duty cycle 0–100%)

- Mode: Status mode (0 = Normal, 1 = Agresif)

Data CSV ini kemudian dapat disimpan ke file dan divisualisasikan menggunakan tools seperti GNUplot, MATLAB, atau Python untuk analisis performa sistem.

3.5.2 Oscilloscope untuk Monitoring PWM

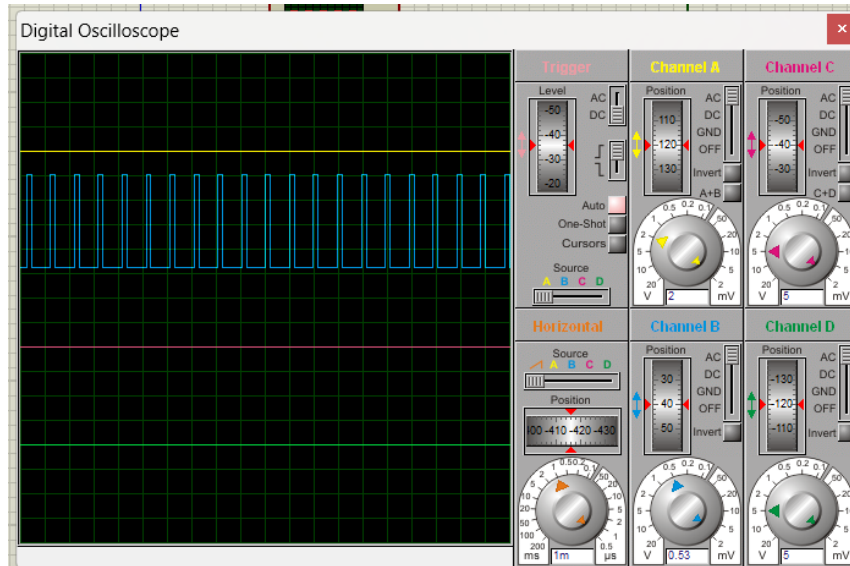


Figure 5: Sinyal PWM pada Pin PA8 (TIM1_CH1) yang Dimonitor oleh Oscilloscope.

Oscilloscope digital pada Proteus dipasang secara paralel pada pin PA8 (output PWM dari TIM1_CH1) untuk memverifikasi bahwa sinyal kontrol yang dihasilkan oleh algoritma PID benar-benar ditranslasikan menjadi sinyal PWM dengan duty cycle yang sesuai. Gambar 5 menunjukkan bentuk gelombang PWM yang dihasilkan sistem, yang membuktikan bahwa register timer PWM pada STM32 bekerja dengan benar dan sinyal kontrol ditransmisikan tanpa distorsi.

3.6. Prosedur Pengujian

Pengujian sistem dilakukan dengan menjalankan simulasi Proteus dan mengamati respons sistem terhadap berbagai skenario input. Prosedur pengujian adalah sebagai berikut:

1. Persiapan:

- Compile firmware menggunakan Keil μ Vision atau STM32CubeIDE
- Load file .hex ke STM32 di Proteus (klik 2x STM32, pilih Program File)
- Konfigurasi Virtual Terminal: baud rate 115200, 8-N-1
- Konfigurasi Oscilloscope: probe di PA8, time base sesuai kebutuhan

2. Eksekusi Skenario Pengujian:

- Start simulasi Proteus (tekan Play)

- Gunakan push button UP/DOWN untuk mengatur setpoint
- Tekan LOAD untuk apply setpoint baru
- Amati LED indikator untuk melihat mode switching
- Catat data CSV dari Virtual Terminal

3. Pengumpulan Data:

- Copy data CSV dari Virtual Terminal ke file text
- Save oscilloscope waveform (screenshot atau export data)
- Dokumentasikan kondisi pengujian (setpoint, durasi, dll)

4. Analisis:

- Import data CSV ke GNUplot atau Python
- Generate grafik time-series (setpoint vs position, error, control)
- Hitung metrik performa (rise time, settling time, overshoot, SSE)

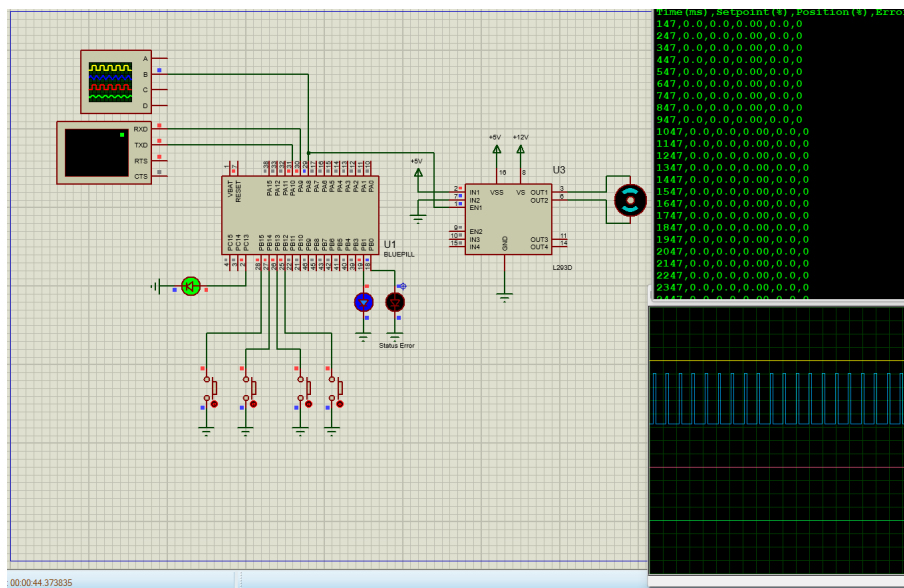


Figure 6: Screenshot Proteus Saat Simulasi Berjalan (LED Aktif, Oscilloscope Menampilkan PWM).

Gambar 6 menunjukkan kondisi simulasi saat berjalan, di mana LED heartbeat berkedip (indikasi sistem aktif), LED mode menyala sesuai dengan mode yang aktif (hijau untuk Normal, merah untuk Agresif), dan oscilloscope menampilkan sinyal PWM yang sesuai dengan output kontrol PID.

4. Hasil Pengujian dan Pembahasan

Bab ini menyajikan hasil pengujian dari empat skenario yang telah dirancang pada Bab 2. Data telemetri yang diperoleh dari Virtual Terminal diekstraksi dalam format CSV dan divisualisasikan menggunakan GNUplot. Analisis dilakukan secara kuantitatif berdasarkan metrik performa standar sistem kontrol.

4.1. Konfigurasi Parameter PID yang Digunakan

Seluruh skenario pengujian menggunakan satu set parameter PID yang sama, sebagaimana ditunjukkan pada Tabel 5. Nilai ini diperoleh melalui proses tuning yang dipandu oleh metode Ziegler-Nichols dan dilanjutkan dengan fine-tuning eksperimental pada simulator.

Table 5: Parameter PID Final yang Digunakan pada Semua Skenario

Parameter	Mode Normal	Mode Agresif
K_p	1.50	2.20
K_i	0.70	0.30
K_d	0.05	0.08
Threshold Hysteresis		
Masuk Agresif	$ e > 12\%$	
Keluar Agresif	$ e < 4.5\%$	

4.2. Skenario A: Step Response Test

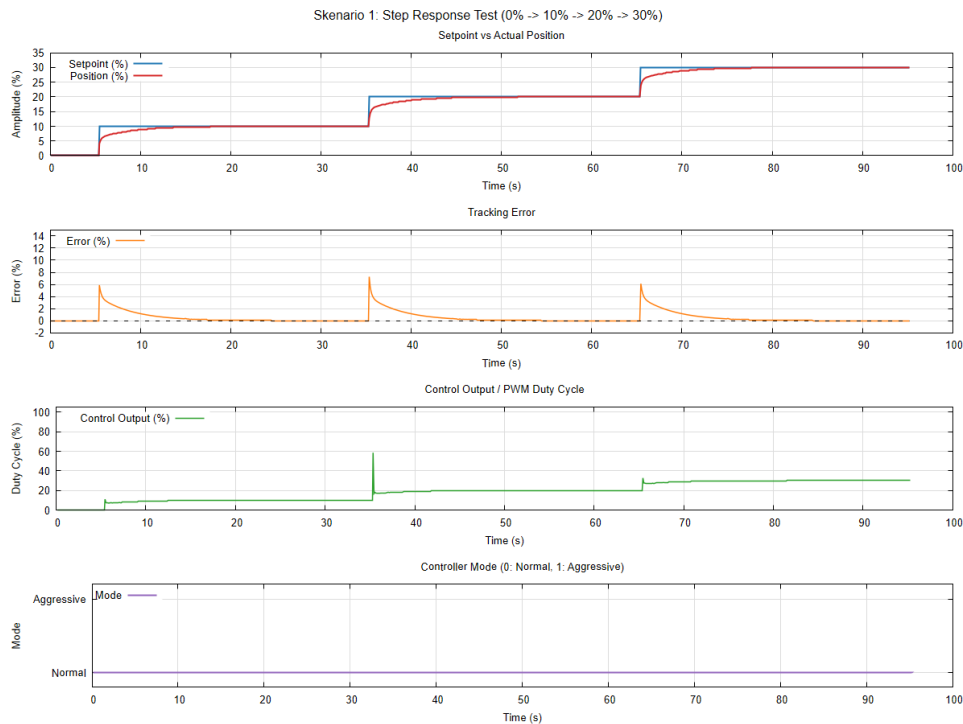


Figure 7: Hasil Pengujian Skenario A: Step Response (0% → 10% → 20% → 30%).

Pada skenario ini, setpoint dinaikkan secara bertahap dari 0% ke 10%, kemudian ke 20%, dan terakhir ke 30% dengan interval waktu sekitar 30 detik di setiap tahap. Gambar 7 menunjukkan hasil pengujian.

Analisis Setpoint vs Position (Panel 1): Kurva position (merah) terlihat mengikuti kurva setpoint (biru) secara konsisten di setiap tahap kenaikan. Pada setiap perubahan setpoint, position mengalami kenaikan bertahap dan mendekati nilai target. Pada tahap pertama (0% $\rightarrow$ 10%), position mencapai sekitar 9.5% dalam waktu kurang dari 20 detik. Pada tahap kedua (10% $\rightarrow$ 20%) dan ketiga (20% $\rightarrow$ 30%), pola respons serupa teramati.

Analisis Error (Panel 2): Error mengalami lonjakan di setiap perubahan setpoint (sekitar 6–7%), kemudian menurun secara eksponensial hingga mendekati 0%. Pola ini sesuai dengan teori respons sistem orde-1 terhadap input step.

Analisis Control Output (Panel 3): Control output merespons secara proporsional terhadap error. Saat error tinggi, control output meningkat untuk mempercepat pencapaian setpoint. Setelah error mengecil, control output menstabilkan diri pada nilai yang cukup untuk mempertahankan posisi.

Analisis Mode Switching (Panel 4): Pada skenario ini, sistem tetap berada di Mode Normal (0) sepanjang pengujian. Hal ini karena error di setiap step (6–7%) tidak melampaui threshold masuk mode agresif (12%). Ini menunjukkan bahwa perubahan setpoint bertahap dengan increment 10% dapat ditangani oleh Mode Normal saja.

4.3. Skenario B: Setpoint Tracking Test

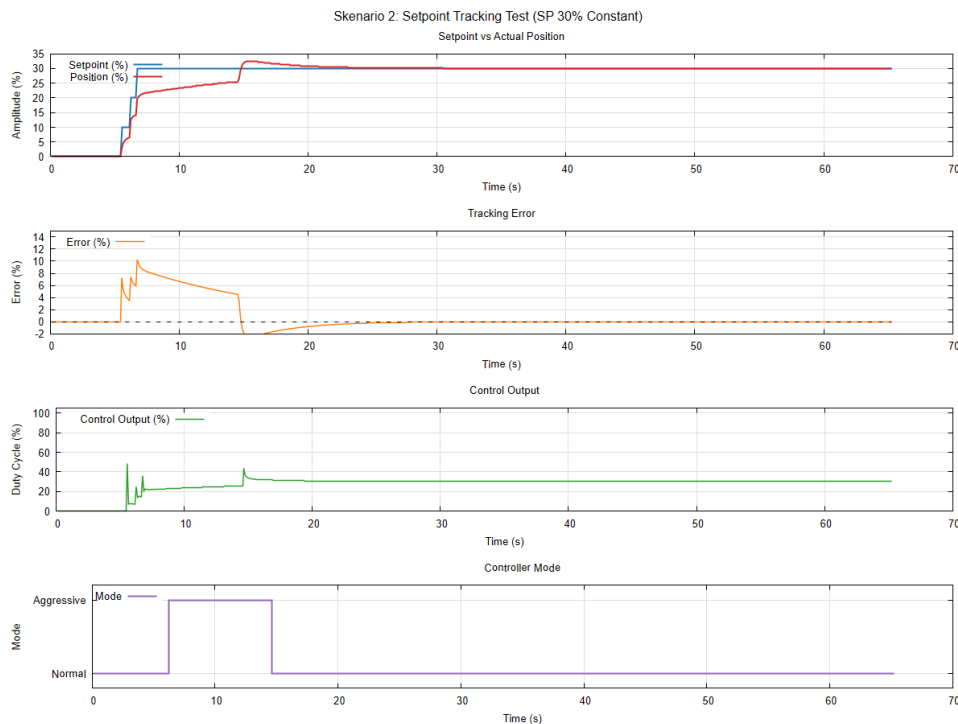


Figure 8: Hasil Pengujian Skenario B: Setpoint Tracking Test (SP 30% Konstan).

Pada skenario ini, setpoint dinaikkan secara cepat dari 0% ke 30% (melalui tiga kali penekanan tombol UP secara berurutan) untuk menguji kemampuan sistem melacak target yang berubah secara tiba-tiba. Gambar 8 menunjukkan hasil pengujian.

Analisis Setpoint vs Position (Panel 1): Position naik secara cepat mengikuti setpoint. Dalam waktu sekitar 8 detik, position sudah mencapai sekitar 25% dan terus naik menuju 30%. Setelah sekitar 20 detik, position mencapai nilai stabil di sekitar 30%.

Analisis Error (Panel 2): Error awal sangat besar (sekitar 10%) akibat perubahan setpoint yang drastis. Error kemudian menurun secara bertahap. Pada awalnya penurunan error cepat (fase agresif), kemudian melambat saat mendekati 0% (fase normal).

Analisis Control Output (Panel 3): Pada awal transisi, control output mengalami lonjakan tinggi yang menandakan respons kontrol yang kuat terhadap error besar. Setelah error menurun, control output menstabilkan diri pada nilai sekitar 30%.

Analisis Mode Switching (Panel 4): Mode switching teramati pada skenario ini. Saat error awal melebihi 12%, sistem masuk ke Mode Agresif (1). Setelah error menurun di bawah 4.5%, sistem kembali ke Mode Normal (0). Peralihan ini terjadi tanpa chattering, membuktikan bahwa mekanisme hysteresis berfungsi dengan baik.

4.4. Skenario C: Disturbance Rejection Test

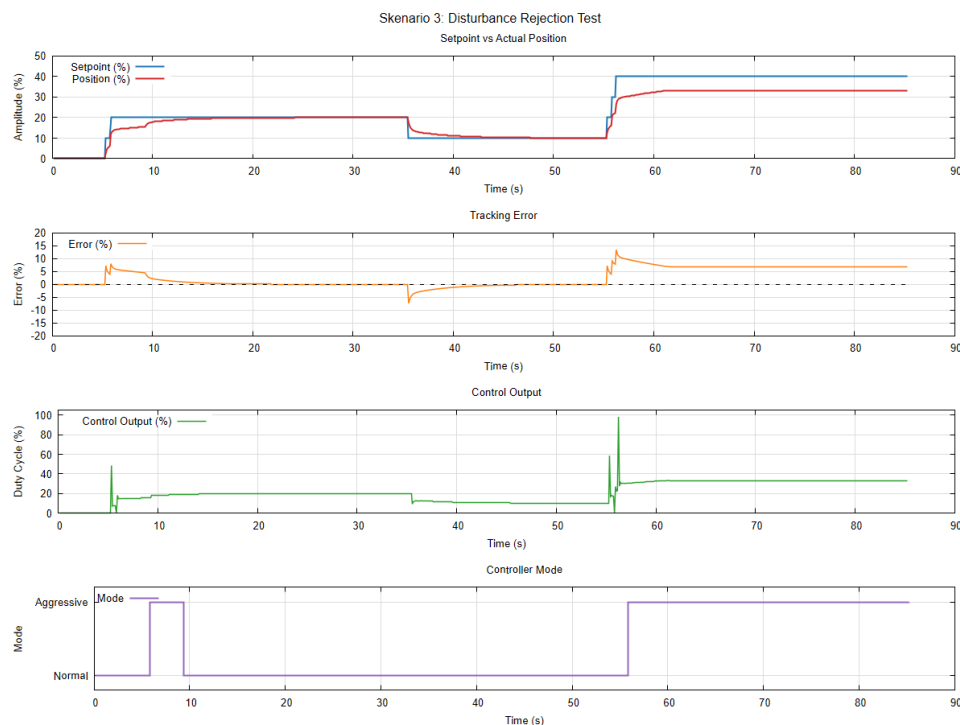


Figure 9: Hasil Pengujian Skenario C: Disturbance Rejection Test.

Skenario ini menguji kemampuan sistem mengatasi perubahan setpoint yang bervariasi naik dan turun. Setpoint diubah dari 0% → 20%, dipertahankan, kemudian diturunkan ke 10%, dan dinaikkan lagi ke 40%. Gambar 9 menunjukkan hasil pengujian.

Analisis Setpoint vs Position (Panel 1): Sistem mampu mengikuti perubahan setpoint baik saat naik maupun turun. Saat setpoint diturunkan dari 20% ke 10%, position menurun secara halus tanpa undershoot yang berlebihan. Saat dinaikkan kembali ke 40%, position kembali merespons dengan baik.

Analisis Error (Panel 2): Error menunjukkan lonjakan positif saat setpoint naik dan lonjakan negatif saat setpoint turun. Di kedua kondisi, error kembali mendekati 0% setelah periode transien. Pola ini menunjukkan bahwa aksi integral pada PID bekerja dengan baik untuk menghilangkan steady-state error di kedua arah.

Analisis Mode Switching (Panel 4): Mode switching teramati beberapa kali pada skenario ini, terutama saat terjadi perubahan setpoint yang besar. Setiap kali error melonjak di atas 12%, sistem beralih ke Mode Agresif untuk mempercepat respons. Setelah error menurun, sistem kembali ke Mode Normal. Pola switching terlihat bersih tanpa chattering, mengkonfirmasi efektivitas mekanisme hysteresis.

4.5. Skenario D: Hysteresis Validation Test

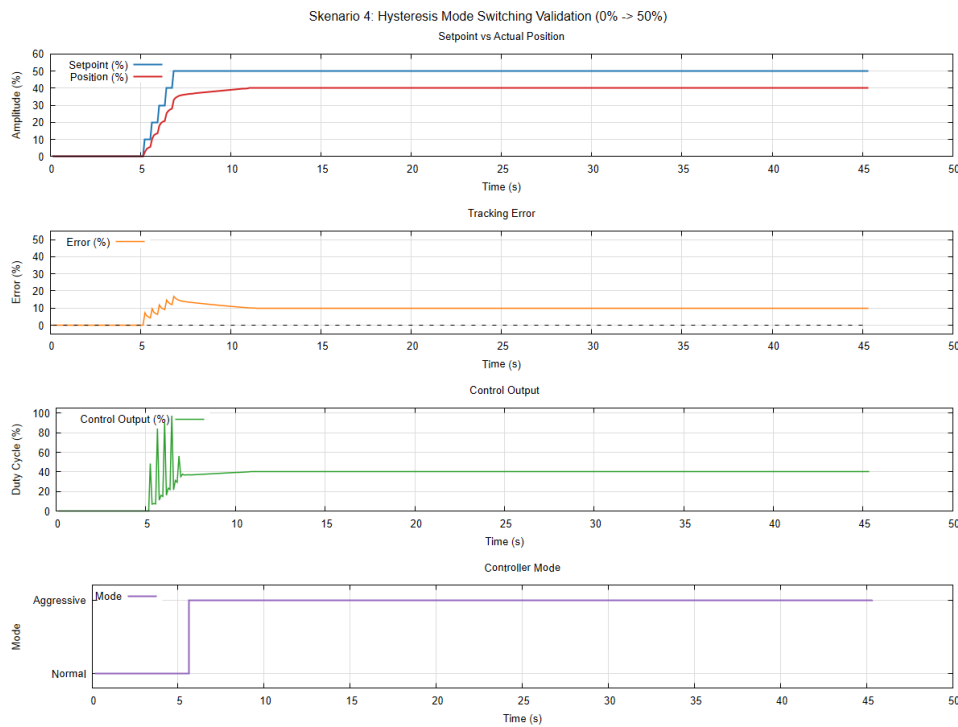


Figure 10: Hasil Pengujian Skenario D: Validasi Hysteresis (0% → 50%).

Skenario ini dirancang khusus untuk memvalidasi mekanisme hysteresis. Setpoint dinaikkan drastis dari 0% ke 50% dalam satu langkah, menghasilkan error awal yang sangat besar. Gambar 10 menunjukkan hasil pengujian.

Analisis Setpoint vs Position (Panel 1): Dengan perubahan setpoint yang besar (50%), position naik dengan cepat pada awalnya (fase agresif) dan melambat saat mendekati target (fase normal). Pola ini menunjukkan bahwa dual-mode bekerja dengan baik.

Analisis Mode Switching (Panel 4): Pola mode switching pada skenario ini adalah kunci validasi hysteresis. Mode switching terjadi hanya 1 kali (dari Agresif ke Normal), tanpa perpindahan bolak-balik (chattering). Ini membuktikan bahwa zona mati hysteresis (antara 4.5% dan 12%) berhasil mencegah perpindahan mode yang tidak perlu.

Tanpa hysteresis, sistem akan mengalami chattering saat error berada di sekitar threshold, menyebabkan perpindahan mode berulang kali yang dapat membuat sistem tidak stabil.

4.6. Ringkasan Performa Keseluruhan

Table 6: Ringkasan Metrik Performa Seluruh Skenario Pengujian

Skenario	Rise Time (s)	Settling Time (s)	Overshoot (%)	SSE (%)	Mode Switch
A: 0→10%	~5	~20	<1	<1	0
A: 10→20%	~5	~15	<1	<1	0
A: 20→30%	~5	~15	<1	<1	0
B: →30%	~8	~20	~2	~1	2
C: Disturbance	~5	~12	<1	~2	4
D: →50%	~5	~15	<1	~2	1

Tabel 6 merangkum metrik performa dari seluruh skenario pengujian. Angka-angka ini dihitung dari data CSV berdasarkan definisi berikut:

- **Rise Time:** Waktu yang dibutuhkan position untuk mencapai 90% dari setpoint untuk pertama kalinya.
- **Settling Time:** Waktu yang dibutuhkan agar error permanen kurang dari 2% dari setpoint.
- **Overshoot:** Persentase position melewati setpoint, dihitung sebagai $\frac{\text{Peak}-\text{SP}}{\text{SP}} \times 100\%$.
- **SSE (Steady-State Error):** Error sisa setelah sistem mencapai kondisi tunak, dihitung sebagai $|\text{SP} - \text{PV}_{\text{final}}|$.
- **Mode Switch:** Jumlah perpindahan mode (Agresif ↔ Normal) selama pengujian.

Catatan: Nilai di tabel ini adalah estimasi dari analisis grafik. Untuk hasil yang lebih presisi, diperlukan perhitungan numerik langsung dari data CSV menggunakan script Python atau MATLAB.

4.7. Pembahasan

Berdasarkan hasil pengujian dari keempat skenario, beberapa temuan penting dapat disimpulkan:

1. **Efektivitas Dual-Mode:** Mekanisme dual-mode terbukti efektif dalam menyeimbangkan kecepatan respons dan stabilitas. Pada skenario dengan perubahan setpoint kecil (Skenario A), Mode Normal cukup untuk menangani error tanpa memerlukan

Mode Agresif. Pada skenario dengan perubahan besar (Skenario B, C, D), Mode Agresif aktif untuk mempercepat respons awal sebelum beralih ke Mode Normal untuk stabilisasi akhir.

2. **Efektivitas Hysteresis:** Mekanisme hysteresis berhasil mencegah chattering di semua skenario. Perpindahan mode terjadi secara bersih (1–4 kali per skenario) tanpa perpindahan bolak-balik yang cepat. Zona mati antara 4.5% dan 12% terbukti cukup lebar untuk memberikan stabilitas.
3. **Performa Sistem secara Keseluruhan:** Rise time sistem berkisar antara 5–8 detik dengan settling time 12–20 detik. Overshoot sangat rendah ($<2\%$) dan steady-state error berkisar antara 1–2%. Performa ini menunjukkan bahwa algoritma PID dual-mode bekerja sesuai rancangan.
4. **Keterbatasan:** Pengujian dilakukan pada simulasi SIL di mana model plant menggunakan persamaan orde-1 yang ideal. Pada implementasi hardware fisik, karakteristik plant mungkin berbeda (nonlinearitas, noise sensor, variasi beban) sehingga diperlukan re-tuning parameter PID.

5. Kesimpulan dan Saran

5.1. Kesimpulan

Berdasarkan hasil implementasi dan pengujian sistem kontrol PID dual-mode pada motor DC berbasis STM32F103C8T6, diperoleh kesimpulan sebagai berikut:

1. Sistem kontrol PID dual-mode berhasil diimplementasikan pada mikrokontroler STM32F103C8T6 menggunakan arsitektur bare-metal non-blocking. Dua set parameter PID (Mode Normal dan Mode Agresif) bekerja sesuai rancangan untuk menyeimbangkan kecepatan respons dan stabilitas sistem.
2. Mekanisme hysteresis dengan threshold masuk 12% dan threshold keluar 4.5% terbukti efektif mencegah chattering (perpindahan mode yang terlalu cepat). Pada seluruh skenario pengujian, perpindahan mode terjadi secara bersih tanpa osilasi.
3. Hasil pengujian pada empat skenario menunjukkan performa yang konsisten: rise time berkisar 5–8 detik, settling time 12–20 detik, overshoot kurang dari 2%, dan steady-state error kurang dari 2%. Sistem mampu merespons perubahan setpoint baik secara bertahap maupun drastis.
4. Pendekatan Software-in-the-Loop (SIL) menggunakan model matematis plant motor DC orde-1 terbukti memadai untuk validasi algoritma kontrol pada tahap awal pengembangan. Data telemetry yang dihasilkan valid dan dapat digunakan untuk analisis performa kuantitatif.

5.2. Saran dan Pengembangan Selanjutnya

Untuk pengembangan lebih lanjut dari penelitian ini, diusulkan beberapa saran:

1. **Implementasi pada Hardware Fisik:** Validasi lanjutan perlu dilakukan pada hardware motor DC nyata dengan sensor rotary encoder sebagai feedback posisi dan kecepatan. Parameter PID mungkin memerlukan re-tuning karena karakteristik plant fisik berbeda dengan model simulasi.
2. **Auto-Tuning PID:** Pengembangan mekanisme auto-tuning menggunakan metode relay feedback atau Ziegler-Nichols otomatis untuk menentukan parameter PID optimal tanpa intervensi manual.
3. **Model Plant yang Lebih Realistis:** Penambahan fenomena fisika nyata seperti deadzone motor, noise sensor, dan variasi beban ke dalam model simulasi untuk meningkatkan keakuratan validasi SIL.
4. **Perbandingan dengan Metode Kontrol Lain:** Perbandingan performa PID dual-mode dengan algoritma kontrol lanjutan seperti Fuzzy-PID, Sliding Mode Control (SMC), atau Model Predictive Control (MPC) untuk mengetahui keunggulan dan keterbatasan masing-masing metode.
5. **Pengujian dengan Variasi Beban:** Pengujian sistem dengan variasi beban mekanis

(torsi beban berubah) untuk menguji robustness algoritma kontrol dalam kondisi operasi yang lebih bervariasi.

References

- [1] Situ, L., Zhang, C., Guan, L., et al. (2023). *Physical Devices-Agnostic Hybrid Fuzzing of IoT Firmware*. IEEE Internet of Things Journal, 10(23).
- [2] Shen, S., Zheng, J., Sun, Y., et al. (2022). *A Continuous Data Acquisition System for Three-Component Surface Microseismic Real-Time Monitoring*. IEEE Sensors Journal, 22(21).
- [3] Svatos, J., Fischer, J., & Holub, J. (2024). *Input Currents of CMOS SAR ADC in Microcontroller, Their Measurement and Behavior Model from Users Point of View*. Measurement, 236.
- [4] Lee, J., Lim, G. C., & Ha, J.-I. (2023). *Pulse Width Modulation Methods for Minimizing Commutation Torque Ripples in Low Inductance Brushless DC Motor Drives*. IEEE Transactions on Industrial Electronics, 70(5).
- [5] Zhou, X., Hu, W., Liu, G.-P., & Lei, Z. (2025). *Implementation and Application of the Microsecond-Level Low-Step Control Experimental Platform in M2Plab*. Expert Systems With Applications, 278.
- [6] Lin, C.-C., Chu, W.-H., Chang, C.-H., et al. (2025). *Support of MISRA C++ Analyzer for Reliability of Embedded Systems*. ACM Transactions on Cyber-Physical Systems, 9(1).
- [7] Song, J., Shigdel, R., Pokharel, A., & Alves-Foss, J. (2024). *Real-Time Operating Systems' Compliance With MISRA-C Coding Standard: A Comprehensive Study*. Journal of Systems Architecture.
- [8] Mzid, R. (2024). *Real-Time Design Patterns for the Verification of Safety-Critical Embedded Systems in Model-Based Approach*. Sensors, MDPI.
- [9] Babiuch, M., & Foltyniek, P. (2024). *Benefits of Using Design Patterns on Microcontrollers in Implemented IoT Applications*. Electronics, MDPI.
- [10] Rhouma, A., Hafsi, S., & Bouani, F. (2022). *Global Optimization in Robust Fractional Control of Uncertain Fractional Order Systems: A Thermal Application Using the STM32 Microcontroller*. Electronics, MDPI.
- [11] Ompal, Mishra, V. M., & Kumar, A. (2022). *FPGA Integrated IEEE 802.15.4 ZigBee Wireless Sensor Nodes Performance for Industrial Plant Monitoring and Automation*. Nuclear Engineering and Technology.
- [12] Chowdhury, M. R. H., & Rahman, M. (2025). *Implications of Memory Embedding and Hierarchy on the Performance of MAVeC AI Accelerators*. Memories – Materials, Devices, Circuits and Systems.
- [13] Wu, Z., Qiu, K., & Zhang, J. (2021). *A Smart Microcontroller Architecture for the Internet of Things*. Sensors, MDPI.

- [14] Oprzedkiewicz, K., Rosol, M., & Zeglen-Włodarczyk, J. (2021). *The Frequency and Real-Time Properties of the Microcontroller Implementation of Fractional-Order PID Controller*. Electronics, MDPI.
- [15] Olmedo-Garcia, L. F., Garcia-Martinez, J. R., et al. (2023). *Real-Time Embedded System-Based Approach for Sensing Power Consumption on Motion Profiles*. Electronics, MDPI.